

Lab 5.2. Calculate the tip

Learn how to write Kotlin code to interact with the UI elements in the tip calculator app, so that you can calculate the tip. You'll also learn how to handle edge cases in your app to make your app more robust.

1. Before you begin

In this codelab, you will be writing code for the tip calculator to go with the UI you created in the previous codelab, [Create XML layouts for Android](#).

Prerequisites

- The code from the [Create XML layouts for Android](#) codelab.
- How to run an Android app from Android Studio in the emulator or on a device.

What you'll learn

- The basic structure of Android apps.
- How to read in values from the UI into your code and manipulate them.
- How to use view binding instead of `findViewById()` to more easily write code that interacts with views.
- How to work with decimal numbers in Kotlin with the `Double` data type.
- How to format numbers as currency.
- How to use string parameters to dynamically create strings.
- How to use **Logcat** in Android Studio to find problems in your app.

What you'll build

- A tip calculator app with a working **Calculate** button.

What you need

- A computer with the latest stable version of [Android Studio](#) installed.
- Starter code for the **Tip Time** app that contains the layout for a tip calculator.

2. Starter app overview

The **Tip Time** app from [the last codelab](#) has all the UI needed for a tip calculator, but no code to calculate the tip. There's a **Calculate** button, but it doesn't work yet. The **Cost of Service** `EditText` allows the user to enter the cost of the service. A list of `RadioButtons` lets the user select the tip percentage, and a `Switch` allows the user to choose whether the tip should be rounded up or not. The tip amount is displayed in a `TextView`, and finally a **Calculate** `Button`

will tell the app to get the data from the other fields and calculate the tip amount. That's where this codelab picks up.

5:41



Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip?



CALCULATE

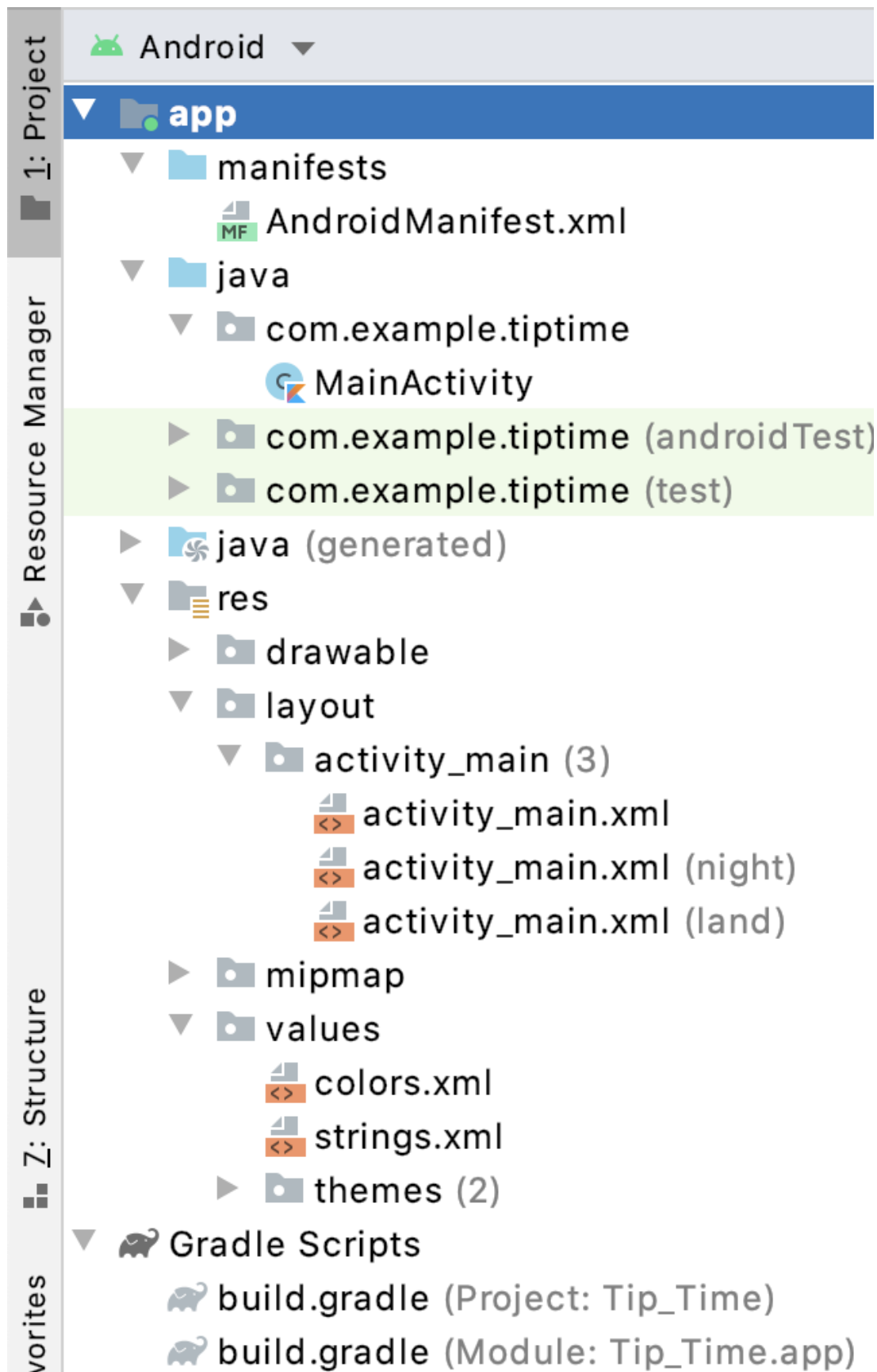
Tip Amount



App project structure

An app project in your IDE consists of a number of pieces, including Kotlin code, XML layouts, and other resources like strings and images. Before making changes to the app, it's good to learn your way around.

1. Open the **Tip Time** project in Android Studio.
2. If the **Project** window isn't showing, select the **Project** tab on the left side of Android Studio.
3. If it's not already selected, choose the **Android** view from the dropdown.



- **java** folder for Kotlin files (or Java files)
- `MainActivity` - class where all of the Kotlin code for the tip calculator logic will go
- **res** folder for app resources
- `activity_main.xml` - layout file for your Android app
- `strings.xml` - contains string resources for your Android app
- **Gradle Scripts** folder

Gradle is the automated build system used by Android Studio. Whenever you change code, add a resource, or make other changes to your app, Gradle figures out what has changed and takes the necessary steps to rebuild your app. It also installs your app in the emulator or physical device and controls its execution.

There are other folders and files involved in building your app, but these are the main ones you'll work with for this codelab and the following ones.

Note: You can also write Android apps in the Java programming language.

3. View binding

In order to calculate the tip, your code will need to access all of the UI elements to read the input from the user. You may recall from earlier codelabs that your code needs to find a reference to a `View` like a `Button` or `TextView` before your code can call methods on the `View` or access its attributes. The Android framework provides a method, `findViewById()`, that does exactly what you need—given the ID of a `View`, return a reference to it. This approach works, but as you add more views to your app and the UI becomes more complex, using `findViewById()` can become cumbersome.

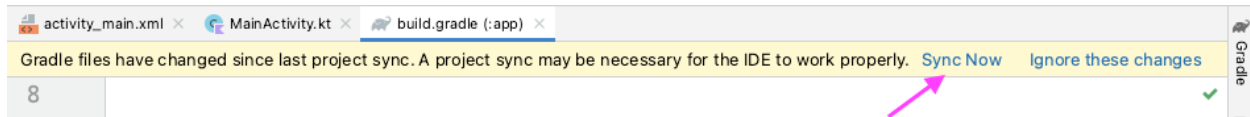
For convenience, Android also provides a feature called [view binding](#). With a little more work up front, view binding makes it much easier and faster to call methods on the views in your UI. You'll need to enable view binding for your app in Gradle, and make some code changes.

Enable view binding

1. Open the app's `build.gradle` file (**Gradle Scripts** > **build.gradle (Module: Tip_Time.app)**)
2. In the `android` section, add the following lines:

```
buildFeatures {
    viewBinding = true
}
```

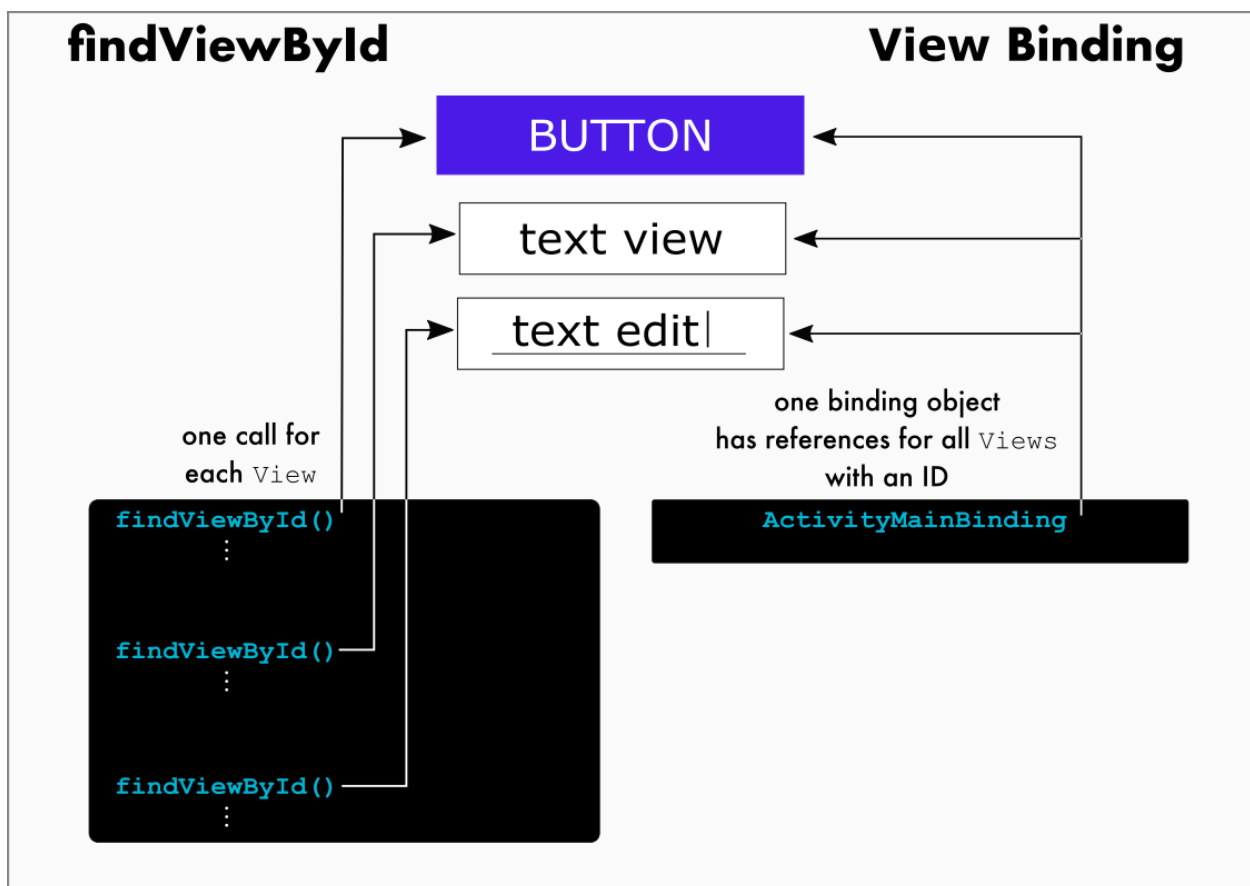
3. Note the message **Gradle files have changed since last project sync.**
4. Press **Sync Now**.



After a few moments, you should see a message at the bottom of the Android Studio window, **Gradle sync finished**. You can close the `build.gradle` file if you want.

Initialize the binding object

In earlier code labs, you saw the `onCreate()` method in the `MainActivity` class. It's one of the first things called when your app starts and the `MainActivity` is initialized. Instead of calling `findViewById()` for each `View` in your app, you'll create and initialize a binding object once.



1. Open `MainActivity.kt` (**app > java > com.example.tiptime > MainActivity**).
2. Replace all of the existing code for `MainActivity` class with this code to setup the `MainActivity` to use view binding:

```
class MainActivity : AppCompatActivity() {
    lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
```

```

        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }
}

```

3. This line declares a top-level variable in the class for the binding object. It's defined at this level because it will be used across multiple methods in `MainActivity` class.

```
lateinit var binding: ActivityMainBinding
```

The `lateinit` keyword is something new. It's a promise that your code will initialize the variable before using it. If you don't, your app will crash.

4. This line initializes the `binding` object which you'll use to access `Views` in the `activity_main.xml` layout.

```
binding = ActivityMainBinding.inflate(layoutInflater)
```

5. Set the content view of the activity. Instead of passing the resource ID of the layout, `R.layout.activity_main`, this specifies the root of the hierarchy of views in your app, `binding.root`.

```
setContentView(binding.root)
```

You may recall the idea of parent views and child views; the root connects to all of them.

Now when you need a reference to a `View` in your app, you can get it from the `binding` object instead of calling `findViewById()`. The `binding` object automatically defines references for every `View` in your app that has an ID. Using view binding is so much more concise that often you won't even need to create a variable to hold the reference for a `View`, just use it directly from the `binding` object.

```

// Old way with findViewById()
val myButton: Button = findViewById(R.id.my_button)
myButton.text = "A button"

// Better way with view binding
val myButton: Button = binding.myButton
myButton.text = "A button"

// Best way with view binding and no extra variable
binding.myButton.text = "A button"

```

How cool is that?

Note: The name of the binding class is generated by converting the name of the XML file to Pascal case and adding the word "Binding" to the end. Similarly, the reference for each view is generated by removing underscores and converting the view name to camel case. For example, in

Pascal case `activity_main.xml` becomes `ActivityMainBinding`, and you can access `@id/text_view` as `binding.textView`.

4. Calculate the tip

Calculating the tip starts with the user tapping the **Calculate** button. This involves checking the UI to see how much the service cost and the percentage tip that the user wants to leave. Using this information you calculate the total amount of the service charge and display the amount of the tip.

Add click listener to the button

The first step is to add a click listener to specify what the **Calculate** button should do when the user taps it.

1. In `MainActivity.kt` in `onCreate()`, after the call to `setContentView()`, set a click listener on the **Calculate** button and have it call `calculateTip()`.

```
binding.calculateButton.setOnClickListener{ calculateTip() }
```

2. Still inside `MainActivity` class but outside `onCreate()`, add a helper method called `calculateTip()`.

```
fun calculateTip() {  
}
```

This is where you'll add the code to check the UI and calculate the tip.

`MainActivity.kt`

```
class MainActivity : AppCompatActivity() {  
  
    lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
        binding.calculateButton.setOnClickListener{ calculateTip() }  
    }  
  
    fun calculateTip() {  
    }  
}
```

Get the cost of service

To calculate the tip, the first thing you need is the cost of service. The text is stored in the `EditText`, but you need it as a number so you can use it in calculations. You may remember the `Int` type from other codelabs, but an `Int` can only hold integers. To use a decimal number in your app, use the data type called `Double` instead of `Int`. You can read more about [numeric data types in Kotlin](#) in the documentation. Kotlin provides a method for converting a `String` to a `Double`, called `toDouble()`.

1. First, get the text for the cost of service. In the `calculateTip()` method, get the text attribute of the **Cost of Service** `EditText`, and assign it to a variable called `stringInTextField`. Remember that you can access the UI element using the binding object, and that you can reference the UI element based on its resource ID name in camel case.

```
val stringInTextField = binding.costOfService.text
```

Notice the `.text` on the end. The first part, `binding.costOfService`, references the UI element for the cost of service. Adding `.text` on the end says to take that result (an `EditText` object), and get the `text` property from it. This is known as *chaining*, and is a very common pattern in Kotlin.

2. Next, convert the text to a decimal number. Call `toDouble()` on `stringInTextField`, and store it in a variable called `cost`.

```
val cost = stringInTextField.toDouble()
```

That doesn't work, though—`toDouble()` needs to be called on a `String`. It turns out that the text attribute of an `EditText` is an `Editable`, because it represents text that can be changed. Fortunately, you can convert an `Editable` to a `String` by calling `toString()` on it.

3. Call `toString()` on `binding.costOfService.text` to convert it to a `String`:

```
val stringInTextField = binding.costOfService.text.toString()
```

Now `stringInTextField.toDouble()` will work.

At this point, the `calculateTip()` method should look like this:

```
fun calculateTip() {  
    val stringInTextField = binding.costOfService.text.toString()  
    val cost = stringInTextField.toDouble()  
}
```

Get the tip percentage

So far you have the cost of the service. Now you need the tip percentage, which the user selected from a `RadioGroup` of `RadioButtons`.

1. In `calculateTip()`, get the `checkedRadioButtonId` attribute of the `tipOptions` `RadioGroup`, and assign it to a variable called `selectedId`.

```
val selectedId = binding.tipOptions.checkedRadioButtonId
```

Now you know which `RadioButton` was selected, one of `R.id.option_twenty_percent`, `R.id.option_eighteen_percent`, or `R.id.fifteen_percent`, but you need the corresponding percentage. You could write a series of `if/else` statements, but it's much easier to use a `when` expression.

2. Add the following lines to get the tip percentage.

```
val tipPercentage = when (selectedId) {  
    R.id.option_twenty_percent -> 0.20  
    R.id.option_eighteen_percent -> 0.18  
    else -> 0.15  
}
```

At this point, the `calculateTip()` method should look like this:

```
fun calculateTip() {  
    val stringInTextField = binding.costOfService.text.toString()  
    val cost = stringInTextField.toDouble()  
    val selectedId = binding.tipOptions.checkedRadioButtonId  
    val tipPercentage = when (selectedId) {  
        R.id.option_twenty_percent -> 0.20  
        R.id.option_eighteen_percent -> 0.18  
        else -> 0.15  
    }  
}
```

Calculate the tip and round it up

Now that you have the cost of service and the tip percentage, calculating the tip is straightforward: the tip is the cost times the tip percentage, `tip = cost of service * tip percentage`. Optionally, that value may be rounded up.

1. In `calculateTip()` after the other code you've added, multiply `tipPercentage` by `cost`, and assign it to a variable called `tip`.

```
var tip = tipPercentage * cost
```

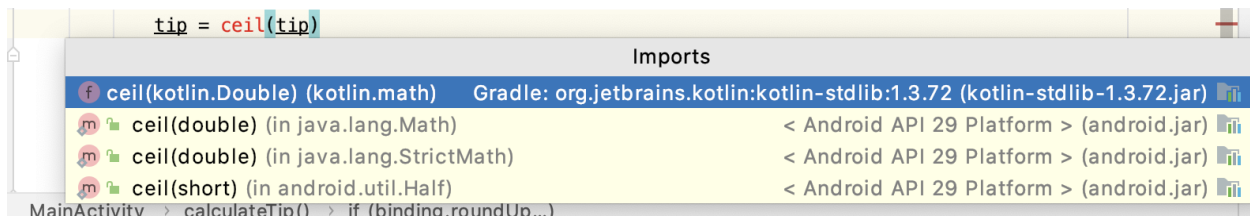
Note the use of `var` instead of `val`. This is because you may need to round up the value if the user selected that option, so the value might change.

For a `Switch` element, you can check the `isChecked` attribute to see if the switch is "on".

2. Assign the `isChecked` attribute of the round up switch to a variable called `roundUp`.

```
val roundUp = binding.roundUpSwitch.isChecked
```

The term rounding means adjusting a decimal number up or down to the closest integer value, but in this case, you only want to round up, or find the ceiling. You can use the `ceil()` function to do that. There are several functions with that name, but the one you want is defined in `kotlin.math`. You could add an `import` statement, but in this case it's simpler to just tell Android Studio which you mean by using `kotlin.math.ceil()`.



If there were several math functions you wanted to use, it would be easier to add an `import` statement.

3. Add an `if` statement that assigns the ceiling of the tip to the `tip` variable if `roundUp` is true.

```
if (roundUp) {  
    tip = kotlin.math.ceil(tip)  
}
```

At this point, the `calculateTip()` method should look like this:

```
fun calculateTip() {  
    val stringInTextField = binding.costOfService.text.toString()  
    val cost = stringInTextField.toDouble()  
    val selectedId = binding.tipOptions.checkedRadioButtonId  
    val tipPercentage = when (selectedId) {  
        R.id.option_twenty_percent -> 0.20  
        R.id.option_eighteen_percent -> 0.18  
        else -> 0.15  
    }  
    var tip = tipPercentage * cost  
    val roundUp = binding.roundUpSwitch.isChecked  
    if (roundUp) {  
        tip = kotlin.math.ceil(tip)  
    }  
}
```

Format the tip

Your app is almost working. You've calculated the tip, now you just need to format it and display it.

As you might expect, Kotlin provides methods for formatting different types of numbers. But the tip amount is a little different—it represents a currency value. Different countries use different currencies, and have different rules for formatting decimal numbers. For example, in U.S. dollars, 1234.56 would be formatted as \$1,234.56, but in Euros, it would be formatted €1.234,56. Fortunately, the Android framework provides methods for formatting numbers as currency, so you don't need to know all the possibilities. The system automatically formats currency based on the language and other settings that the user has chosen on their phone. Read more about [NumberFormat](#) in the Android developer documentation.

1. In `calculateTip()` after your other code, call `NumberFormat.getCurrencyInstance()`.

```
NumberFormat.getCurrencyInstance()
```

This gives you a number formatter you can use to format numbers as currency.

2. Using the number formatter, chain a call to the `format()` method with the `tip`, and assign the result to a variable called `formattedTip`.

```
val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
```

3. Notice that `NumberFormat` is drawn in red. This is because Android Studio can't automatically figure out which version of `NumberFormat` you mean.
4. Hover the pointer over `NumberFormat`, and choose **Import** in the popup that appears.

```
val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
```

Unresolved reference: NumberFormat

Import ↗ ↶ ↷ More actions... ↗ ↶

5. In the list of possible imports, choose **NumberFormat (java.text)**. Android Studio adds an `import` statement at the top of the `MainActivity` file, and `NumberFormat` is no longer red.

Display the tip

Now it's time to display the tip in the tip amount `TextView` element of your app. You could just assign `formattedTip` to the `text` attribute, but it would be nice to label what the amount represents. In the U.S. with English, you might have it display **Tip Amount: \$12.34**, but in other languages the number might need to appear at the beginning or even the middle of the string. The

Android framework provides a mechanism for this called *string parameters*, so someone translating your app can change where the number appears if needed.

1. Open `strings.xml` (**app > res > values > strings.xml**)
2. Change the `tip_amount` string from `Tip Amount` to `Tip Amount: %s`.

```
<string name="tip_amount">Tip Amount: %s</string>
```

The `%s` is where the formatted currency will be inserted.

3. Now set the text of the `tipResult`. Back in the `calculateTip()` method in `MainActivity.kt`, call `getString(R.string.tip_amount, formattedTip)` and assign that to the `text` attribute of the tip result `TextView`.

```
binding.tipResult.text = getString(R.string.tip_amount, formattedTip)
```

At this point, the `calculateTip()` method should look like this:

```
fun calculateTip() {
    val stringInTextField = binding.costOfService.text.toString()
    val cost = stringInTextField.toDouble()
    val selectedId = binding.tipOptions.checkedRadioButtonId
    val tipPercentage = when (selectedId) {
        R.id.option_twenty_percent -> 0.20
        R.id.option_eighteen_percent -> 0.18
        else -> 0.15
    }
    var tip = tipPercentage * cost
    val roundUp = binding.roundUpSwitch.isChecked
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
    binding.tipResult.text = getString(R.string.tip_amount, formattedTip)
}
```

You're almost there. When developing your app (and viewing the preview), it's useful to have a placeholder for that `TextView`.

4. Open `activity_main.xml` (**app > res > layout > activity_main.xml**).
5. Find the `tip_result` `TextView`.
6. Remove the line with the `android:text` attribute.

```
android:text="@string/tip_amount"
```

7. Add a line for the `tools:text` attribute set to `Tip Amount: $10`.

```
tools:text="Tip Amount: $10"
```

Because this is just a placeholder, you don't need to extract the string into a resource. It won't appear when you run your app.

8. Note that the tools text appears in the **Layout Editor**.
9. Run your app. Enter an amount for the cost and select some options, then press the **Calculate** button.

5:41



Tip Time

52

How was the service?

☐ Amazing (20%)

☒ Good (18%)

☐ Okay (15%)

Round up tip?



CALCULATE

Tip Amount: \$9.36



Congratulations—it works! If you are not getting the correct tip amount, go back to step 1 of this section and ensure you've made all the necessary code changes.

5. Test and debug

You've run the app at various steps to make sure it does what you want, but now it's time for some additional testing.

For now, think about how the information moves through your app in the `calculateTip()` method, and what could go wrong at each step.

For example, what would happen in this line:

```
val cost = stringInTextField.toDouble()
```

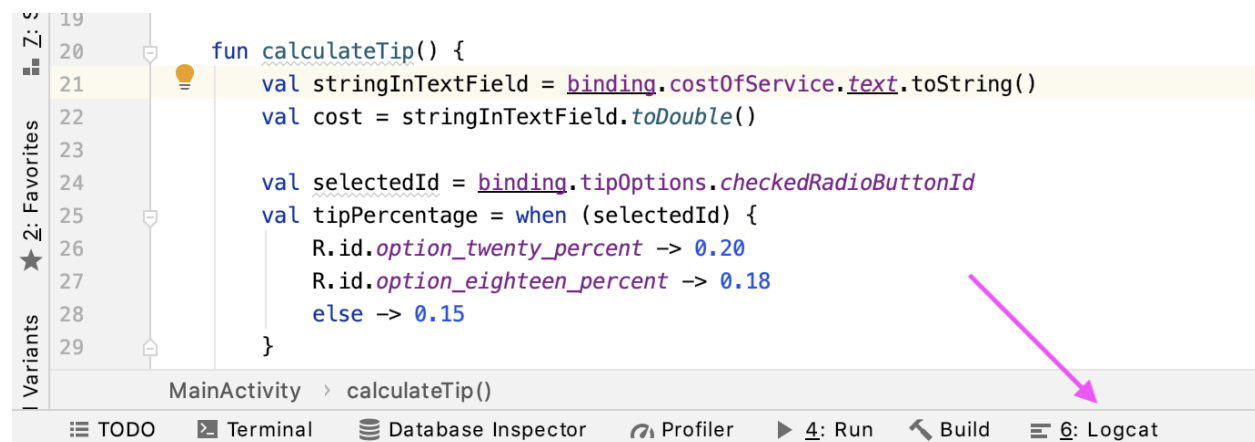
if `stringInTextField` didn't represent a number? What would happen if the user didn't enter any text and `stringInTextField` was empty?

1. Run your app in the emulator, but instead of using **Run > Run 'app'**, use **Run > Debug 'app'**.
2. Try some different combinations of cost, tip amount, and rounding up the tip or not and verify that you get the expected result for each case when you tap **Calculate**.
3. Now try deleting all the text in the **Cost of Service** field and tap **Calculate**. Uh, oh...your program crashes.

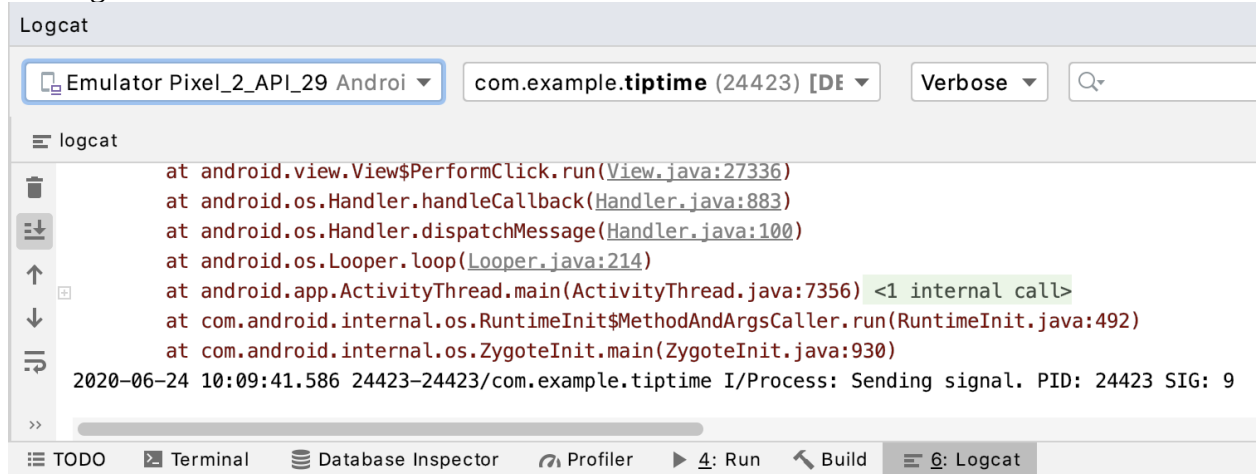
Debug the crash

The first step in dealing with a bug is to find out what happened. Android Studio keeps a [log](#) of what is happening in the system which you can use to find out what went wrong.

1. Press the **Logcat** button at the bottom of the Android Studio, or choose **View > Tool Windows > Logcat** in the menus.



2. The **Logcat** window appears at the bottom of Android Studio, filled with some strange-looking text.



The text is a *stack trace*, a list of which methods were being called when the crash occurred.

3. Scroll upward in the **Logcat** text until you find a line which includes the text **FATAL EXCEPTION**.

```
2020-06-24 10:09:41.564 24423-24423/com.example.tiptime E/AndroidRuntime:
FATAL EXCEPTION: main
    Process: com.example.tiptime, PID: 24423
    java.lang.NumberFormatException: empty String
        at
    sun.misc.FloatingDecimal.readJavaFormatString(FloatingDecimal.java:1842)
        at sun.misc.FloatingDecimal.parseDouble(FloatingDecimal.java:110)
        at java.lang.Double.parseDouble(Double.java:538)
        at com.example.tiptime.MainActivity.calculateTip(MainActivity.kt:22)
        at
    com.example.tiptime.MainActivity$onCreate$1.onClick(MainActivity.kt:17)
```

4. Read downward until you find the line with `NumberFormatException`.

```
java.lang.NumberFormatException: empty String
```

To the right it says `empty String`. The type of the exception tells you it was something to do with a number format, and the rest tells you the basis of the problem: an empty `String` was found when it should have been a `String` with a value.

5. Continue reading downward, and you'll see some calls to `parseDouble()`.
6. Below those calls, find the line with `calculateTip`. Note that it includes your `MainActivity` class, too.

```
at com.example.tiptime.MainActivity.calculateTip(MainActivity.kt:22)
```

7. Look carefully at that line, and you can see exactly where in your code the call was made, line 22 in `MainActivity.kt`. (If you typed your code differently, it may be a different number.) That line converts the `String` to a `Double` and assigns the result to the `cost` variable.

```
val cost = stringInTextField.toDouble()
```

8. Look in the Kotlin documentation for the `toDouble()` method that works on a `String`. The method is referred to as [String.toDouble\(\)](#).
9. The page says "Exceptions: `NumberFormatException` - if the string is not a valid representation of a number."

An *exception* is the system's way of saying there is a problem. In this case, the problem is that `toDouble()` couldn't convert the empty `String` into a `Double`. Even though the `EditText` has `inputType=numberDecimal`, it's still possible to enter some values that `toDouble()` can't handle, like an empty string.

Learn about null

Calling `toDouble()` on a string that is empty or a string that doesn't represent a valid decimal number doesn't work. Fortunately Kotlin also provides a method called `toDoubleOrNull()` which handles these problems. It returns a decimal number if it can, or it returns `null` if there's a problem.

Null is a special value that means "no value". It's different from a `Double` having a value of `0.0` or an empty `String` with zero characters, `""`. `Null` means there is no value, no `Double` or no `String`. Many methods expect a value and may not know how to handle `null` and will stop, which means the app crashes, so Kotlin tries to limit where `null` is used. You'll learn more about this in future lessons.

Your app can check for `null` being returned from `toDoubleOrNull()` and do things differently in that case so the app doesn't crash.

1. In `calculateTip()`, change the line that declares the `cost` variable to call `toDoubleOrNull()` instead of calling `toDouble()`.

```
val cost = stringInTextField.toDoubleOrNull()
```

2. After that line, add a statement to check if `cost` is `null`, and if so to return from the method. The `return` instruction means exit the method without executing the rest of the instructions. If the method needed to return a value, you would specify it with a `return` instruction with an expression.

```
if (cost == null) {  
    return  
}
```

3. Run your app again.
4. With no text in the **Cost of Service** field, tap **Calculate**. This time your app doesn't crash! Good job—you found and fixed the bug!

Handle another case

Not all bugs will cause your app to crash—sometimes the results might be potentially confusing to the user.

Here's another case to consider. What will happen if the user:

1. enters a valid amount for the cost
2. taps **Calculate** to calculate the tip
3. deletes the cost
4. taps **Calculate** again?

The first time the tip will be calculated and displayed as expected. The second time, the `calculateTip()` method will return early because of the check you just added, but the app will still show the previous tip amount. This might be confusing to the user, so add some code to clear the tip amount if there's a problem.

1. Confirm this problem is what happens by entering a valid cost and tapping **Calculate**, then deleting the text, and tapping **Calculate** again. The first tip value should still be displayed.
2. Inside the `if` just added, before the `return` statement, add a line to set the `text` attribute of `tipResult` to an empty string.

```
if (cost == null) {  
    binding.tipResult.text = ""  
    return  
}
```

This will clear the tip amount before returning from `calculateTip()`.

3. Run your app again, and try the above case. The first tip value should go away when you tap **Calculate** the second time.

Congratulations! You've created a working tip calculator app for Android and handled some edge cases!

6. Adopt good coding practices

Your tip calculator works now, but you can make code a little better and make it easier to work with in the future by adopting good coding practices.

1. Open `MainActivity.kt` (**app > java > com.example.tiptime > MainActivity**).

2. Look at the beginning of the `calculateTip()` method, and you might see that it is underlined with a wavy, grey line.

```
20 fun calculateTip() {  
21     val stringInTextField = binding.costOfService.text.toString()  
22     val cost = stringInTextField.toDoubleOrNull()  
23     if (cost == null) {
```

3. Hover the pointer over `calculateTip()`, you'll see a message, **Function 'calculateTip' could be private** with a suggestion below to **Make 'calculateTip' 'private'**.

```
20 fun calculateTip() {  
21     va  
22     va  
23     if  
24  
25  
26 }  
27  
28 va  
29 val tipPercentage = when (selectedId) {
```

Function 'calculateTip' could be private

Make 'calculateTip' 'private' More actions...

com.example.tiptime.MainActivity
public final fun calculateTip(): Unit

Tip_Time.app

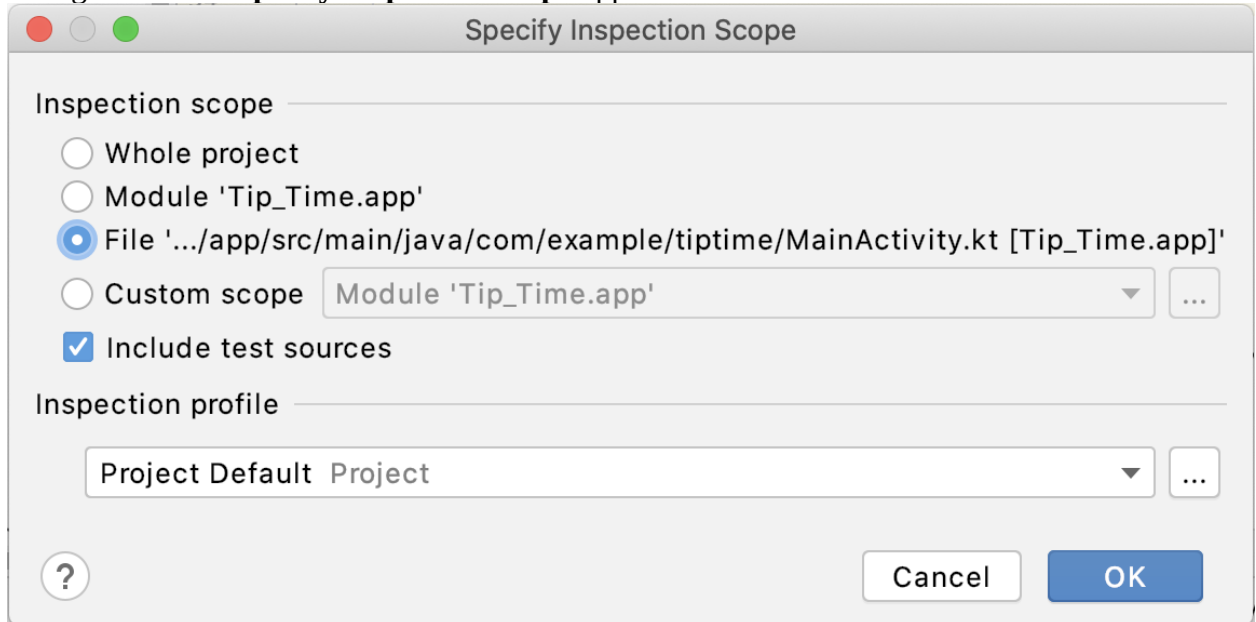
Recall from earlier codelabs that `private` means the method or variable is only visible to code within that class, in this case, `MainActivity` class. There's no reason for code outside `MainActivity` to call `calculateTip()`, so you can safely make it `private`.

4. Choose **Make 'calculateTip' 'private'**, or add the `private` keyword before `fun calculateTip()`. The grey line under `calculateTip()` disappears.

Inspect the code

The grey line is very subtle and easy to overlook. You could look through the whole file for more grey lines, but there's a simpler way to make sure you find all of the suggestions.

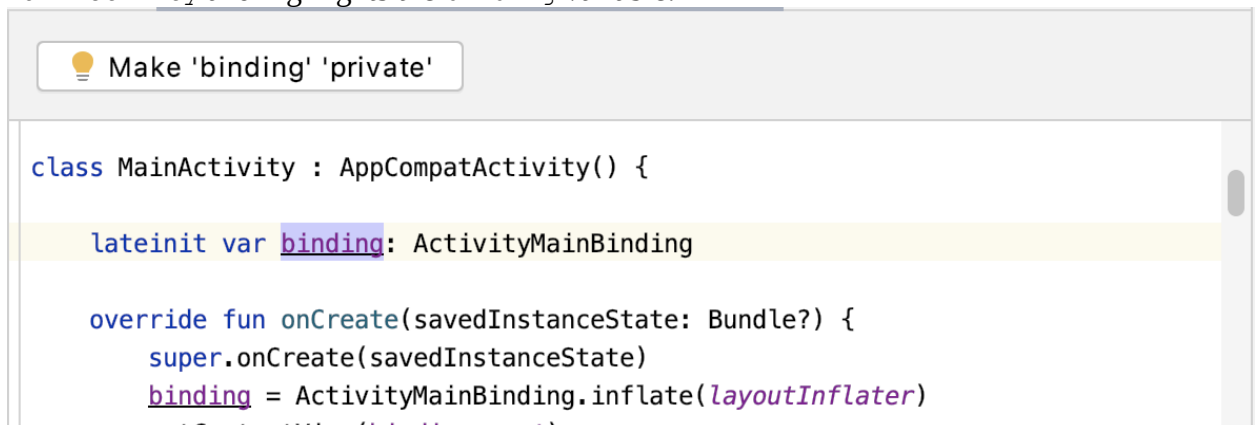
1. With `MainActivity.kt` still open, choose **Analyze > Inspect Code...** in the menus. A dialog box called **Specify Inspection Scope** appears.



2. Choose the option that starts with **File** and press **OK**. This will limit the inspection to just `MainActivity.kt`.
3. A window with **Inspection Results** appears at the bottom.
4. Click on the grey triangles next to **Kotlin** and then next to **Style issues** until you see two messages. The first says **Class member can have 'private' visibility**.



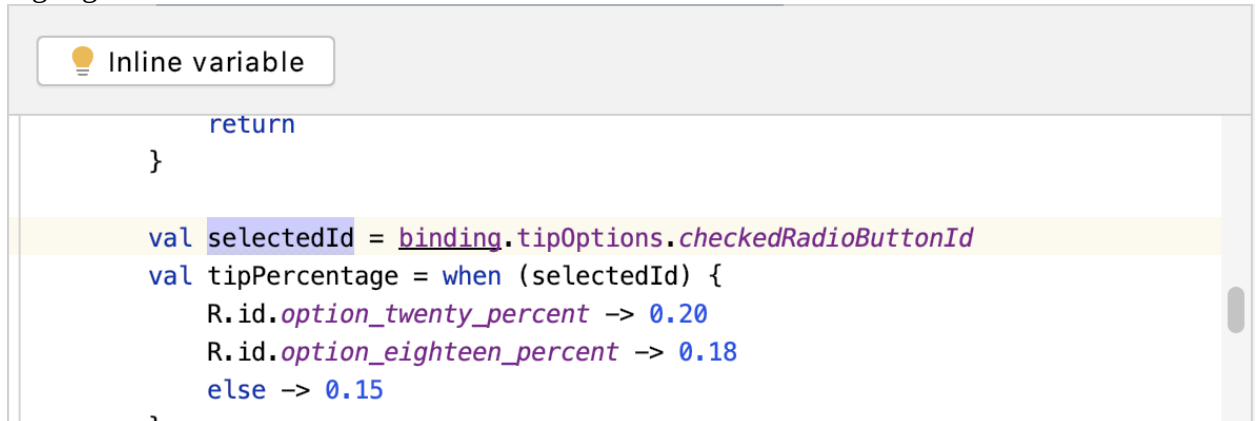
5. Click on the grey triangles until you see the message **Property 'binding' could be private** and click on the message. Android Studio displays some of the code in `MainActivity` and highlights the binding variable.



6. Press the **Make 'binding' 'private'** button. Android Studio removes the issue from the **Inspection Results**.
7. If you look at `binding` in your code, you'll see that Android Studio has added the keyword `private` before the declaration.

```
private lateinit var binding: ActivityMainBinding
```

8. Click on the gray triangles in the results until you see the message **Variable declaration could be inlined**. Android Studio again displays some of the code, but this time it highlights the `selectedId` variable.



9. If you look at your code, you'll see that `selectedId` is only used twice: first in the highlighted line where it is assigned the value of `tipOptions.checkedRadioButtonId`, and in the next line in the `when`.
10. Press the **Inline variable** button. Android Studio replaces `selectedId` in the `when` expression with the value that was assigned in the line before. And then it removes the previous line completely, because it's no longer needed!

```

val tipPercentage = when (binding.tipOptions.checkedRadioButtonId) {
    R.id.option_twenty_percent -> 0.20
    R.id.option_eighteen_percent -> 0.18
    else -> 0.15
}

```

That's pretty cool! Your code has one less line, and one less variable.

Removing unnecessary variables

Android Studio doesn't have any more results from the inspection. However, if you look at your code closely, you'll see a similar pattern to what you just changed: the `roundUp` variable is assigned on one line, used on the next line, and not used anywhere else.

1. Copy the expression to the right of the `=` from the line where `roundUp` is assigned.

```
val roundUp = binding.roundUpSwitch.isChecked
```

2. Replace `roundUp` in the next line with the expression you just copied,
`binding.roundUpSwitch.isChecked.`

```
if (binding.roundUpSwitch.isChecked) {  
    tip = kotlin.math.ceil(tip)  
}
```

3. Delete the line with `roundUp`, because it isn't needed anymore.

You just did the same thing that Android Studio helped you do with the `selectedId` variable. Again your code has one less line and one less variable. These are small changes, but they help make your code more concise and more readable.

(Optional) Eliminate repetitive code

Once your app is running correctly, you can look for other opportunities to clean up your code and make it more concise. For example, when you don't enter a value in cost of service, the app updates `tipResult` to be an empty string `""`. When there is a value, you use `NumberFormat` to format it. This functionality can be applied elsewhere in the app, for example, to display a tip of `0.0` instead of the empty string.

To reduce duplication of very similar code, you can extract these two lines of code to their own function. This helper function can take as input a tip amount as a `Double`, formats it, and updates the `tipResult` `TextView` on screen.

1. Identify the duplicated code in `MainActivity.kt`. These lines of code could be used multiple times in the `calculateTip()` function, once for the `0.0` case, and once for the general case.

```
val formattedTip = NumberFormat.getCurrencyInstance().format(0.0)  
binding.tipResult.text = getString(R.string.tip_amount, formattedTip)
```

2. Move the duplicated code to its own function. One change to the code is to take a parameter `tip` so that the code works in multiple places.

```
private fun displayTip(tip : Double) {  
    val formattedTip = NumberFormat.getCurrencyInstance().format(tip)  
    binding.tipResult.text = getString(R.string.tip_amount, formattedTip)  
}
```

3. Update your `calculateTip()` function to use the `displayTip()` helper function and check for `0.0`, too.

`MainActivity.kt`

```
private fun calculateTip() {  
    ...
```

```

        // If the cost is null or 0, then display 0 tip and exit this
function early.
        if (cost == null || cost == 0.0) {
            displayTip(0.0)
            return
        }

        ...
        if (binding.roundUpSwitch.isChecked) {
            tip = kotlin.math.ceil(tip)
        }

        // Display the formatted tip value on screen
        displayTip(tip)
    }

```

Note

Even though the app is functioning now, it's not ready for production yet. You need to do more testing. And you need to add some visual polish and follow Material Design guidelines. You'll also learn to change the app theme and app icon in the following codelabs.

7. Solution code

The solution code for this codelab is below.

5:32



Tip Time

100

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ OK (15%)

Round up tip?



CALCULATE

Tip Amount: \$20.00



MainActivity.kt

(note on the first line: replace the package name if yours is different than com.example.tiptime)

```
package com.example.tiptime

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.example.tiptime.databinding.ActivityMainBinding
import java.text.NumberFormat

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.calculateButton.setOnClickListener { calculateTip() }
    }

    private fun calculateTip() {
        val stringInTextField = binding.costOfService.text.toString()
        val cost = stringInTextField.toDoubleOrNull()
        if (cost == null) {
            binding.tipResult.text = ""
            return
        }

        val tipPercentage = when (binding.tipOptions.checkedRadioButtonId) {
            R.id.option_twenty_percent -> 0.20
            R.id.option_eighteen_percent -> 0.18
            else -> 0.15
        }

        var tip = tipPercentage * cost
        if (binding.roundUpSwitch.isChecked) {
            tip = kotlin.math.ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
        binding.tipResult.text = getString(R.string.tip_amount, formattedTip)
    }
}
```

Modify strings.xml

```
<string name="tip_amount">Tip Amount: %s</string>
```

Modify activity_main.xml

```

...
<TextView
    android:id="@+id/tip_result"
    ...
    tools:text="Tip Amount: $10" />
...

```

Modify the app module's `build.gradle`

```

android {
    ...

    buildFeatures {
        viewBinding = true
    }
    ...
}

```

8. Summary

- View binding lets you more easily write code that interacts with the UI elements in your app.
- The `Double` data type in Kotlin can store a decimal number.
- Use the `checkedRadioButtonId` attribute of a `RadioGroup` to find which `RadioButton` is selected.
- Use `NumberFormat.getCurrencyInstance()` to get a formatter to use for formatting numbers as currency.
- You can use string parameters like `%s` to create dynamic strings that can still be easily translated into other languages.
- Testing is important!
- You can use **Logcat** in Android Studio to troubleshoot problems like the app crashing.
- A stack trace shows a list of methods that were called. This can be useful if the code generates an exception.
- Exceptions indicate a problem that code didn't expect.
- `Null` means "no value."
- Not all code can handle `null` values, so be careful using it.
- Use **Analyze > Inspect Code** for suggestions to improve your code.

9. More Codelabs to improve your UI

Great job on getting the tip calculator to work! You'll notice that there are still ways to improve the UI to make the app look more polished. If you're interested, check out these additional codelabs to learn more about how to change the app theme and app icon, as well as how to follow best practices from the Material Design guidelines for the Tip Time app!

- [Change the app color theme](#)

- [Change the app icon](#)
- [Create a more polished user experience](#)

10. Learn more

- `Double` data type in Kotlin
- [Numeric data types](#) in Kotlin
- [Null Safety](#) in Kotlin
- [App Manifest](#)
- View [binding](#)
- `NumberFormat.getCurrencyInstance()`
- [string parameters](#)
- [testing](#)
- [Logcat](#)
- [Analyze a stack trace](#)

11. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab.

- With the unit converter app for cooking in the previous practice, add code for the logic and calculations to convert units like milliliters to or from fluid ounces.